ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)

Software Design Report

SWE 4404: Software Project Lab II

Project Title

CodingLab

A Coding Practice Platform for University Students

Team Members

Name	Student ID
Rakibur Rahman Ananta	230042128
SM Raiyan Al Arafat	230042134
Nafian Nazat Mahid	230042144

Group-10

Supervisor

Samia Nawsheen

Junior Lecturer, Department of CSE, IUT

Contents

1 Introduction **2**

2 Software Design Architecture Diagram **2**

3 Data Flow Diagram (DFD) **4**

4 Database Design **5**

5 Database Schema Design (ER Diagram) **6**

6 Process Flow Diagrams **7**
6.1 Problem Approval Flow 8
6.2 Code Execution Flow 9
6.3 Role Change Request Flow 10
6.4 Contest Management Flow 11

7 Functional Requirements Summary **12**

8 Non-Functional Requirements (NFRs) **12**

9 Conclusion **12**

1 Introduction

CodingLab is a web-based coding practice platform designed specifically for university students. It provides a structured environment where students can solve programming problems aligned with their academic curriculum, receive instant automated verdicts, and track their learning progress over time.

The platform supports three primary user roles:

- **Student** – Solves problems, submits code, tracks progress, and competes on leaderboards.
- **Problem Setter** – Creates problems with test cases, submits for admin approval, and monitors submission analytics.
- **Admin** – Moderates problems, manages users and roles, organizes contests, and oversees the platform dashboard.

This report presents the software design of CodingLab through a collection of diagrams that illustrate the system architecture, data flow, and key processes. As the system does not follow an Object-Oriented design approach, this report includes a Software Design Architecture Diagram, a Data Flow Diagram (DFD), a Database Design overview, a full Database Schema (ER) Diagram, and four supporting Process Flow Diagrams. The system follows a three-layer architecture with a Node.js/Express.js backend, React.js frontend, and MongoDB database.

2 Software Design Architecture Diagram

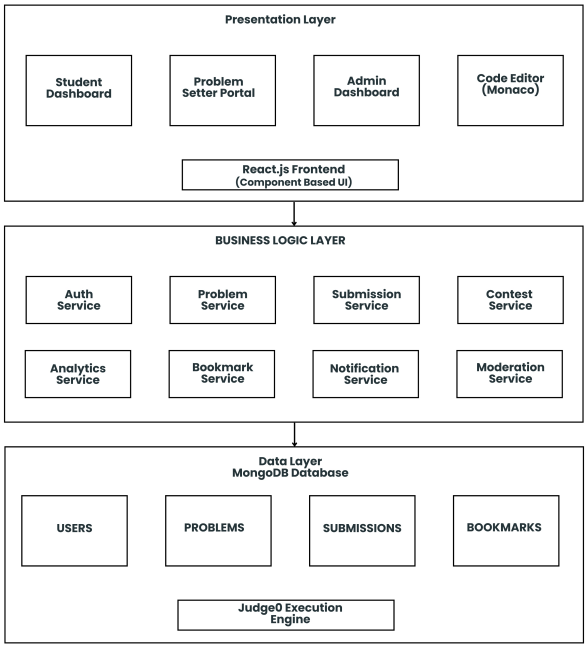


Figure 1: Three-Layer Architecture Diagram

Explanation of Architecture Diagram

The system follows a three-layer architecture with clear separation of concerns:

Presentation Layer (Frontend):

- Built with React.js for component-based UI development.
- Includes role-specific interfaces: Student Dashboard, Problem Setter Portal, and Admin Dashboard.
- Features a Monaco Editor (VS Code engine) for in-browser code editing.
- Communicates with the backend via REST API calls.

Business Logic Layer (Backend):

- Built with Node.js and Express.js framework.
- **Auth Service** – Manages user registration, login, and role-based access control.
- **Problem Service** – Handles problem creation, editing, and retrieval.
- **Submission Service** – Processes code submissions and interacts with JDoodle.
- **Contest Service** – Manages contest creation, registration, and real-time ranking.
- **Moderation Service** – Handles problem approval/rejection workflow.
- **Analytics Service** – Aggregates submission statistics for problem setters.
- **Notification Service** – Manages in-app notifications
- **Bookmark Service** – Manages student bookmarks for problems.

Data Layer (Storage):

- **MongoDB** – NoSQL document store with flexible schemas for Users, Problems, Submissions, Contests, and Bookmarks.
- **jdoodle** – Runs on Third-Party online Server
- **Cloudinary** – Media storage for optional video solution links.

3 Data Flow Diagram (DFD)

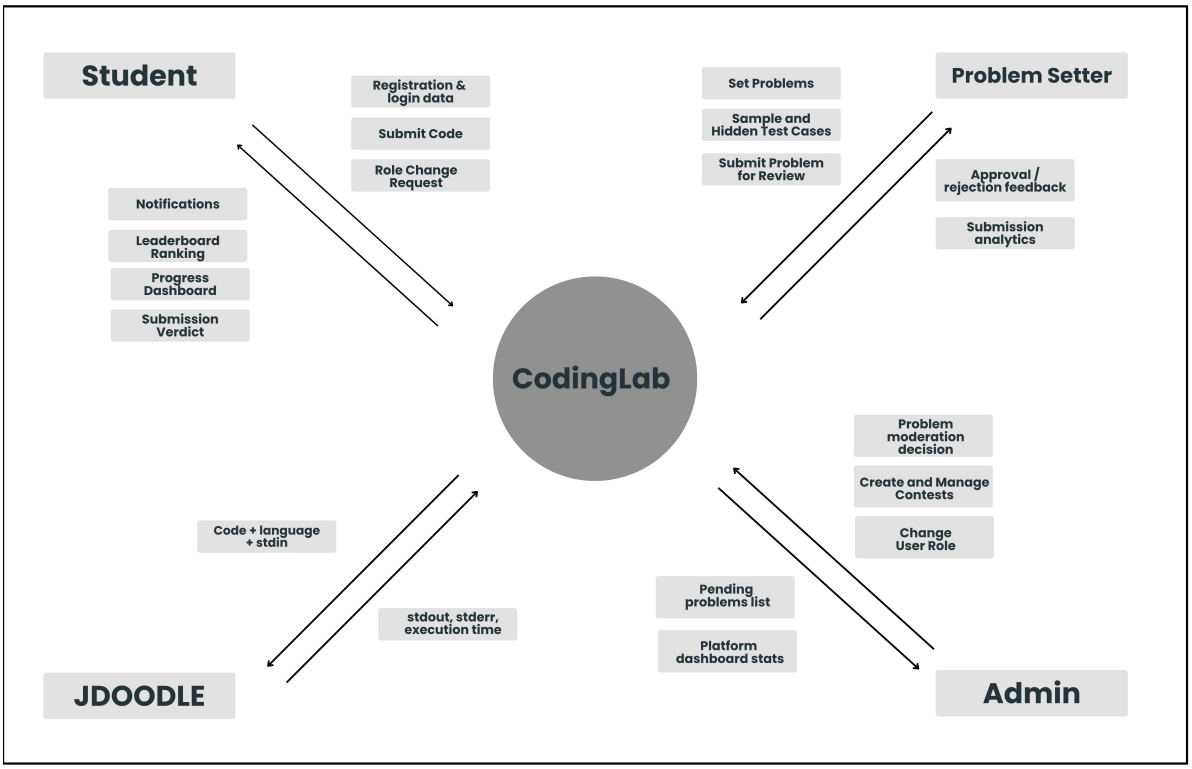


Figure 2: Context/Level 0 Data Flow Diagram

Explanation of Data Flow Diagram

The Context Level DFD provides a high-level overview of the entire CodingLab system, showing how data flows between the system and its three external entities: **Student**, **Problem Setter**, and **Admin**.

From → To	Data Flow	Description
Student → System	Registration & Login	User authentication data
Student → System	Code + stdin	code submissions for evaluation
System → Student	Verdict	AC, WA, TLE, RE, CE results
System → Student	Dashboard	Progress tracking and statistics
Problem Setter → System	Problem Data	Title, description, test cases
Problem Setter → System	Review Request	Submit problem for admin approval
System → Problem Setter	Feedback	Approval/rejection with comments
Admin → System	Moderation	Approve/reject problems
Admin → System	Contest Data	Create and manage contests
System → Admin	Dashboard	Platform statistics and pending items

4 Database Design

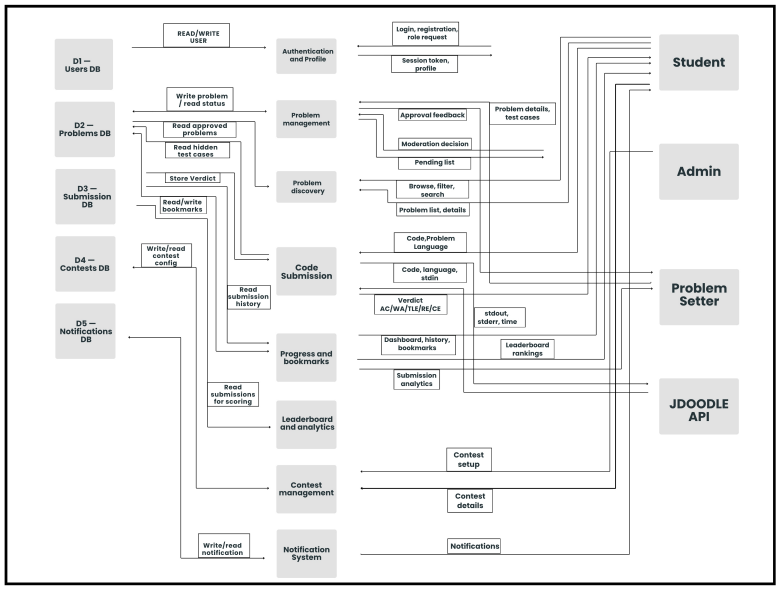


Figure 3: MongoDB Database Collections and Data Flows

Explanation of Database Design

The system uses MongoDB, with the following collections representing the core data entities referenced throughout the DFD and process flow diagrams:

D1 – Users DB: Stores all user accounts with authentication credentials and profile information. Supports role-based access (Student, Problem Setter, Admin) and manages session tokens and role change requests. *Key Data:* name, email, passwordHash, role, joinDate, lastLogin.

D2 – Problems DB: Stores all problems with their details, test cases, and status. Supports problem creation, editing, and moderation workflow, and manages sample test cases (visible to students) and hidden test cases (used for evaluation). *Key Data:* title, description, constraints, difficulty, topicTags, testCases, status, setterId.

D3 – Submissions DB: Stores all code submissions with their verdicts and execution details. Tracks submission history for students and supports leaderboard calculations and analytics. *Key Data:* problemId, userId, code, verdict, executionTime, submittedAt.

D4 – Contests DB: Stores contest configurations including selected problems and time windows. Manages contest registrations and rankings. *Key Data:* title, startTime, endTime, problemIds, registrations, rankings.

5 Database Schema Design (ER Diagram)

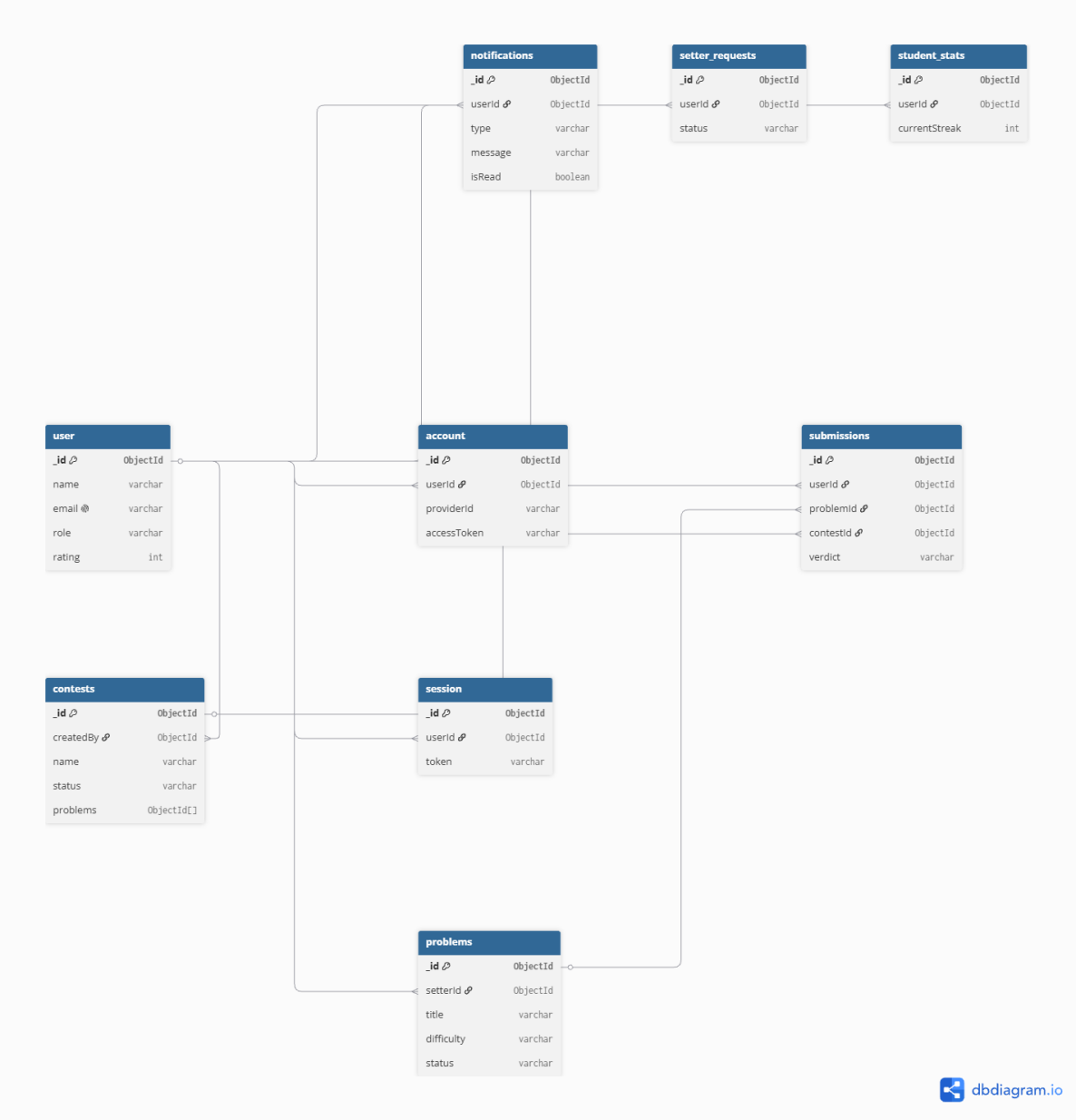


Figure 4: MongoDB Database Schema and Entity Relationships

Explanation of Database Schema Design

CodingLab's data layer is implemented in MongoDB across nine core collections, trimmed here to their essential relationships.

user is the central entity – every other collection references it. It stores the account's role (student/setter/admin) and rating.

account and **session** support authentication (via Better Auth), linking OAuth credentials and login tokens back to a `user`.

problems are created by a `user` acting as a setter (`setterId`), and carry a moderation `status` (pending/approved/rejected) that drives the approval workflow.

submissions is the busiest collection – it links a `user` (who submitted), a `problem` (what was solved), and optionally a `contest` (if submitted during a contest window), storing the resulting `verdict`.

contests are created by a `user` acting as admin, and embed a `problems` reference array directly (no separate junction table), consistent with MongoDB's document-oriented design rather than relational normalization.

notifications are generated per `user` for events like role-change approval or verdict delivery

setter_requests and **student_stats** are both one-to-one extensions of a `user` – the former tracking a pending role-change application, the latter tracking a student's solved-problem streak and progress metrics.

Note on Relationship Modeling

Since MongoDB is a document-oriented NoSQL database rather than a relational database, all relationships shown are implemented as *reference fields* (ObjectId pointers resolved via Mongoose's `.populate()`), not enforced foreign key constraints. The diagram reflects intended data relationships at the application level, not database-level constraints.

6 Process Flow Diagrams

6.1 Problem Approval Flow

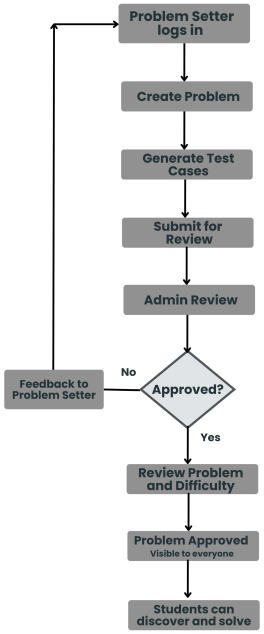


Figure 5: Problem Approval Flow

This diagram illustrates the complete lifecycle of a problem from creation to publication:

1. **Problem Creation** – The Problem Setter logs in and creates a new problem with title, description, input/output format, constraints, difficulty, topic tags, and test cases.
2. **Test Case Generation** – The system generates test cases (both sample and hidden) based on the problem constraints.
3. **Submit for Review** – The Problem Setter submits the completed problem for admin review.
4. **Admin Review** – The Admin reviews the problem for quality, correctness, and appropriateness.
5. **Decision** – If rejected, the Admin provides feedback and the Problem Setter can revise and resubmit. If approved, the Admin confirms the problem difficulty.
6. **Publication** – Approved problems become visible to all students, who can then discover and solve them.

6.2 Code Execution Flow

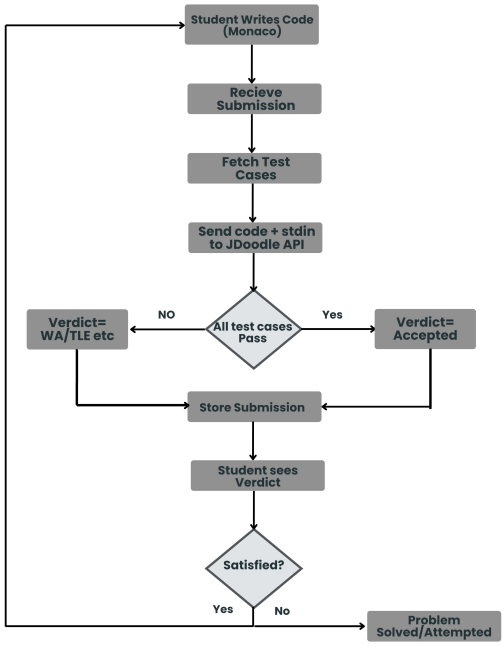


Figure 6: Code Execution Flow

This diagram illustrates how a student's code submission is processed through the system:

1. **Code Writing** – Student writes code using the Monaco editor in the browser.
2. **Submission** – The frontend sends the code string to the backend via a REST API call.
3. **Test Case Fetch** – The backend fetches all hidden test cases for the problem from the database.
4. **Execution** – The backend passes the code and test cases to the Jdoodle API. Jdoodle compiles the code and runs it against each test case.
5. **Evaluation** – If all test cases pass, the verdict is Accepted (AC). If any test case fails, the verdict is Wrong Answer (WA). Other possible verdicts include Time Limit Exceeded (TLE), Runtime Error (RE), or Compilation Error (CE).
6. **Storage** – The verdict and submission details are stored in the D3 Submissions database.
7. **Display** – The student sees the verdict on screen. If unsatisfied, they can modify and resubmit their code.

6.3 Role Change Request Flow

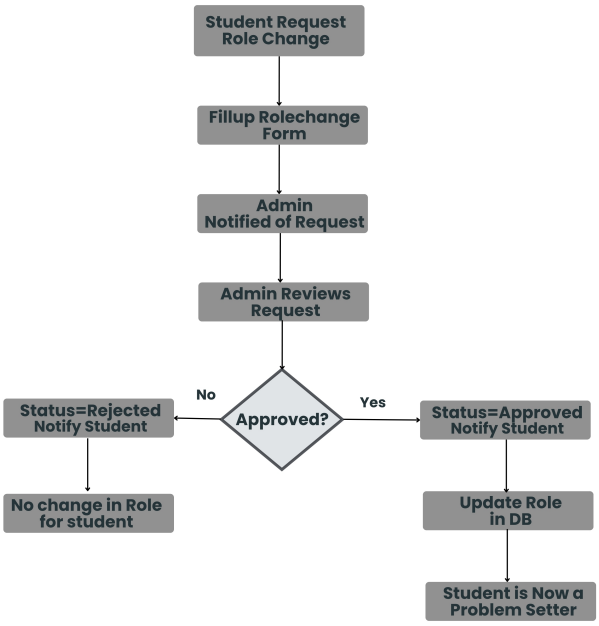


Figure 7: Role Change Request Flow

This diagram illustrates how a student can request to become a Problem Setter:

1. **Request Initiation** – A student who wants to contribute content requests a role change through the platform.
2. **Form Submission** – The student fills out a role change form with necessary information.
3. **Admin Notification** – The admin receives a notification about the pending request.
4. **Admin Review** – The admin reviews the student's request and qualifications.
5. **Decision** – If approved, the student's role is updated to 'Problem Setter' in the database. If rejected, the student's role remains unchanged.
6. **Notification** – The student is notified of the decision via in-app notification and email.

6.4 Contest Management Flow

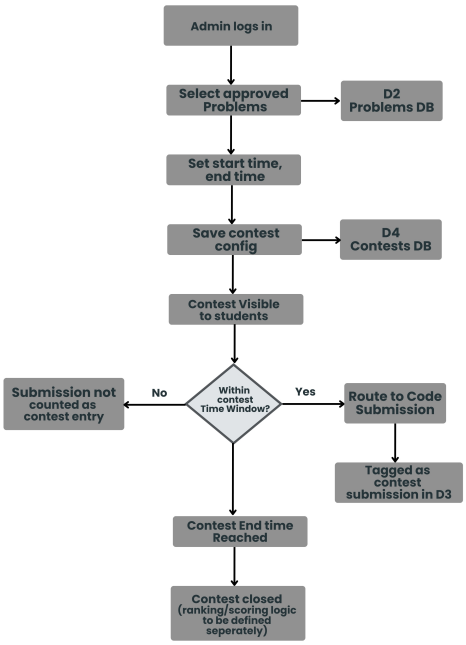


Figure 8: Contest Management Flow

This diagram illustrates how admins create and manage coding contests:

1. **Admin Login** – Admin logs into the platform.
2. **Problem Selection** – Admin selects a set of approved problems from the Problems DB (D2).
3. **Configure Contest** – Admin sets start time, end time, and duration for the contest.
4. **Save Configuration** – Contest details are saved in the Contests DB (D4).
5. **Publish Contest** – The contest becomes visible to all students.
6. **Student Participation** – Students can view and participate in the contest during the time window.
7. **Submission Processing** – If a submission is made within the contest time window, it is tagged as a contest submission and routed to the code execution flow. If outside the time window, the submission is not counted as a contest entry.
8. **Contest End** – When the end time is reached, the contest is closed and rankings are calculated.

7 Functional Requirements Summary

Based on the diagrams and system design, the key functional requirements of CodingLab are:

FR ID	Functional Requirement	Priority
FR-01	User Registration and Authentication	High
FR-02	Problem Discovery and Browsing	High
FR-03	In-Browser Code Editor	Medium
FR-04	Code Execution and Instant Verdicts	High
FR-05	Progress Tracking Dashboard	Medium
FR-08	Leaderboard	Low
FR-09	Problem Creation and Management	High
FR-10	Problem Submission and Review Workflow	High
FR-11	Admin Moderation	High
FR-15	Contest Management	Low
FR-16	Notification System	Medium
FR-17	Role Change Requests	High

8 Non-Functional Requirements (NFRs)

NFR ID	Requirement	Description
NFR-01	Performance	Code execution verdict must be returned within 5 seconds
NFR-02	Scalability	System must support concurrent submissions from 100+ users
NFR-03	Security	Passwords must be hashed; sessions auto-timeout after 30 minutes
NFR-04	Usability	UI must be intuitive with clear navigation for all roles
NFR-05	Reliability	System uptime must be 99.9% during contest hours
NFR-06	Extensibility	Modular architecture supporting future languages (Java, Python)

9 Conclusion

The design of CodingLab follows established software engineering principles with a three-layer architecture that cleanly separates presentation, business logic, and data storage. The system is represented through eight key diagrams:

1. **Software Design Architecture Diagram** – Shows the overall system structure.
2. **Data Flow Diagram (DFD)** – Illustrates data flow between entities and the system.

3. **Database Design Diagram** – Shows the logical MongoDB data stores referenced in the DFD.
4. **Database Schema Design (ER Diagram)** – Shows the full implemented MongoDB collections and their relationships.
5. **Problem Approval Flow** – Illustrates the problem lifecycle.
6. **Code Execution Flow** – Shows how student submissions are processed.
7. **Role Change Request Flow** – Illustrates the role change workflow.
8. **Contest Management Flow** – Shows how contests are created and managed.

The system design addresses the key functional requirements of CodingLab, providing a comprehensive solution for university students learning programming.